

Frontend Engineering: Day 4

DOM, HTML DOM, Events, Elements, APIs

Sayan Chowdhury

Engineer- Data Analytics

Design Automation

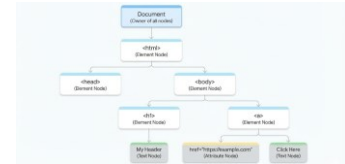
D-O-M (Document Object Model)

Think of it as the bridge between your web page (HTML) and your programming language (JavaScript).

When a web page is loaded, the browser creates a "Model" of the page. This model is a tree-like structure that represents every single part of the HTML document as an Object.



```
<!DOCTYPE html>
<html>
  <head>
    </head>
  <body>
    <h1>My Header</h1>
    <a href="https://example.com">Click Here</a>
  </body>
</html>
```

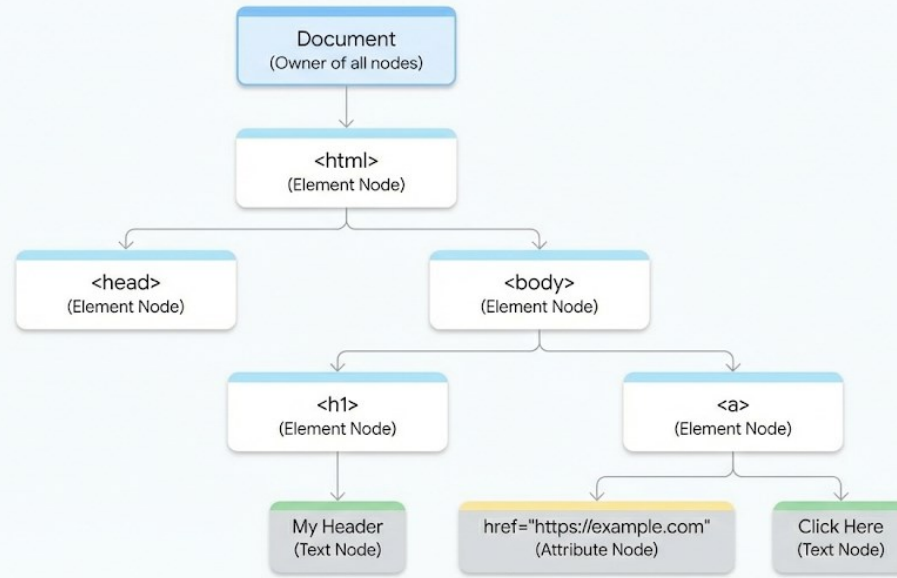


D-O-M (Document Object Model)

Think of it as the bridge between your web page (HTML) and your programming language (JavaScript).

When a web page is loaded, the browser creates a "Model" of the page. This model is a tree-like structure that represents every single part of the HTML document as an Object.

```
<!DOCTYPE html>
<html>
  <head>
  </head>
  <body>
    <h1>My Header</h1>
    <a href="https://example.com">Click Here</a>
  </body>
</html>
```



D-O-M (Document Object Model)

Why do we need it?

Without the DOM, JavaScript would just see a long string of text. Because of the DOM, JavaScript can "talk" to the elements on the page.

With the DOM, JavaScript can:

- Change any HTML element (like changing the text of a <p> tag).
- Change any HTML attribute (like changing an src of an image).
- Change CSS styles (like changing the background colour).
- Remove existing elements or Add new ones.
- React to events (like a button click or a mouse hover).

HTML vs. DOM: What's the difference?

HTML: Is the text file you write (index.html). It is static.

DOM: Is the live representation of that file in the browser's memory. If you use JavaScript to change a button colour, the HTML file doesn't change, but the DOM does.

HTML DOM



```
<html>↵
<body>↵
↵
<p id="demo"></p>↵
↵
<script>↵
  // Access a paragraph Element↵
  const myPara = document.getElementById("demo");↵
  // Change the content of the Element↵
  myPara.innerHTML = "Hello World!";↵
</script>↵
↵
</body>↵
</html>↵
```

document is the **HTML document**



getElementById() is a **document method**.



myPara = getElementById("demo") retrieves the **"demo"** element.



innerHTML is an **element property**.



myPara.innerHTML = "Hello World!" changes the **property**.

HTML DOM: Selecting Elements

Developers use global objects like `document` and `window` as entry points to any API.

If you want to access any element in an HTML page, you always start with accessing the `document` object. The `document` object represents your web page.

To manipulate HTML with JavaScript, you first need to select an element.

Method	Description
<code>document.getElementById(id)</code>	Find an element by element id
<code>document.getElementsByTagName(name)</code>	Find elements by tag name
<code>document.getElementsByClassName(name)</code>	Find elements by class name
<code>document.querySelector(selector)</code>	Find the first element that matches a CSS selector
<code>document.querySelectorAll(selector)</code>	Find all elements that match a CSS selector

Adding Event Handlers

Method	Description
<code>document.getElementById(id).onclick = function(){code}</code>	Adding event handler code to an onclick event

HTML DOM: Accessing Element Content

Method	Description
<i>element.innerHTML</i>	The HTML content of an element
<i>element.textContent</i>	The text content of an element
<i>element.attribute</i>	Change the attribute value of an HTML element
<i>element.style.property</i>	The style of an HTML element
<i>element.setAttribute()</i>	Create or set a new attribute

Manipulating Structure

Method	Description
<i>document.createElement()</i>	Creates a new HTML element
<i>document.removeChild()</i>	Remove an HTML element
<i>document.appendChild()</i>	Add an HTML element
<i>document.replaceChild()</i>	Replace an HTML element

HTML DOM

The easiest way to find an HTML element in the DOM, is by using the element id.

If the element is found, the method will return the element as an object (in element).

If the element is not found, element will contain null.



```
const element = document.getElementById("intro");
```

You can find all elements with a particular tag



```
const element = document.getElementsByTagName("p");
```


HTML DOM

If you want to find all HTML elements with the same class name, use *getElementsByClassName()*



```
const x = document.getElementsByClassName("intro");
```

If you want to find all HTML elements by CSS Selectors



```
const myPara = document.querySelector(".demo");
```

HTML DOM

The `querySelectorAll()` Method : If you want to find all HTML elements that match a specified CSS selector (id, class names, types, attributes, values of attributes, etc), use the `querySelectorAll()` method.



```
const x = document.querySelectorAll("p.intro");
```

HTML DOM

The innerHTML Property

The easiest way to get the content of an element is by using the innerHTML property. The innerHTML property is useful for getting or replacing the content of HTML elements.



```
<html>
<body>

<p id="p1">Hello World!</p>

<script>
document.getElementById("p1").innerHTML = "New text!";
</script>
</body>
</html>
```

HTML DOM

Changing an Attribute

To change the value of an HTML attribute, use this syntax:

`document.getElementById(id).attribute = new value`

```

<!DOCTYPE html>
<html>
<body>

<script>
document.getElementById("myImage").src = "landscape.jpg";
</script>
</body>
</html>
```

HTML DOM

Changing an Attribute

To change the value of an HTML attribute, use this syntax:

```
document.getElementById(id).attribute = new value
```



```
<!DOCTYPE html>
<html>
<body>
<script>
document.getElementById("demo").innerHTML = "Date: " + Date();
</script>
</body>
</html>
```

HTML DOM

Changing HTML Style

The HTML DOM allows JavaScript to change the style of HTML elements. To change the style of an HTML element, use this syntax

`document.getElementById(id).style.property= new style`

```
<html>
<body>

<p id="p2">Hello World!</p>

<script>
document.getElementById("p2").style.color="blue";
</script>

</body>
</html>
```

HTML DOM

Using Events

The HTML DOM allows you to execute code when an event occurs.

Events are generated by the browser when "things happen" to HTML elements:

- An element is clicked on
- The page has loaded
- Input fields are changed

```
<!DOCTYPE html>
<html>
<body>

<h1 id="id1">My Heading 1</h1>

<button type="button"
onclick="document.getElementById( 'id1' ).style.color = 'red'">
Click Me!</button>

</body>
</html>
```

HTML DOM

JavaScript Events

Often, when events happen, you may want to do something.

When JavaScript is used in HTML pages, JavaScript can react on events.

JavaScript lets you execute code when events are detected.

HTML allows event handler attributes, with JavaScript code, to be added to HTML elements.

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

HTML DOM

DOMContentLoaded

The DOMContentLoaded event fires when the browser has fully loaded the HTML and built the Document Object Model (DOM) tree, but has not necessarily finished loading external resources like images and stylesheets.

The DOMContentLoaded event is best for initializing the user interface, attaching event handlers, and performing actions that only require the DOM to be ready.

```
<p id="demo"></p>

<script>
// Add Event Listener to document
document.addEventListener("DOMContentLoaded", function() {
  document.getElementById("demo").innerHTML = "HTML is loaded!";
});
</script>
```

HTML DOM

Window Load

The load event fires when the entire page has fully loaded, including all dependent resources such as images, stylesheets, and sub-frames.

The load event is best for actions that require all resources available, such as getting the dimensions of an image or checking the browser type.

```
<p id="demo"></p>
<script>
  // Add Event Listener to window
  window.addEventListener("load", function() {
    document.getElementById("demo").innerHTML = "Page is fully loaded!";
  });
</script>
```

LocalStorage

The localStorage Object

The localStorage object stores the data with no expiration date. The data will not be lost when the browser is closed, and will be available the next day, week, or year.

Use the **localStorage.setItem()** method to create name/value pairs

Use the **localStorage.getItem()** method to retrieve the values set



```
localStorage.setItem("lastname", "Smith");  
localStorage.getItem("lastname");
```

sessionStorage

The sessionStorage Object

The sessionStorage object is equal to the localStorage object, except that it stores the data for only one session! The data is deleted when the user closes the specific browser tab.

Using API's

```
@app.route('/api/data', methods=['GET'])
def get_data():
    return jsonify({
        "name": "Sayan",
        "status": "Success",
        "message": "Data received from Flask!"
    })
```

```
fetch('http://127.0.0.1:5000/api/data')
    .then(response => response.json()) // Convert response to JSON
    .then(data => {
        console.log(data.message); // "Data received from Flask!"
    })
    .catch(error => console.error('Error:', error))
```

Understanding the "Promise"

Why do we use .then()?

- Network requests take time (milliseconds to seconds).
- JavaScript is Asynchronous: It doesn't stop the whole page while waiting.
- A Promise is a placeholder for a value that will arrive in the future.
- Success: .then() runs.
- Failure: .catch() runs (e.g., if the server is down).

Connecting to the DOM

Once you have the data from Flask, use the DOM to show it to the user.



```
.then(data => {  
  ...document.getElementById("title").innerText = data.name;  
})
```

Mini Project1

With Onclick

A Counter with Reset and Save

In this project you will build a counter with buttons to:

- Increase the counter
- Decrease the counter
- Reset the counter
- Save the counter
- Load the counter

Mini Project1

With Event Listeners

A Counter with Reset and Save

In this project you will build a counter with buttons to:

- Increase the counter
- Decrease the counter
- Reset the counter
- Save the counter
- Load the counter

Thank You